Guía de Laboratorio N°4

Herencia, Clases Abstractas, Polimorfismo,

Docente: Camilo Alejandro Fuentes Beals – PhD
Programación Científica

1. Ejercicios

Instrucciones Generales

Cree un archivo IPYNB (Jupyter Notebook) con los ejercicios separados por celdas.
El archivo debe tener el siguiente formato de nombre: NOMBRE_APELLIDO.
Incluye docstrings en cada clase y método.
Cada ejercicio debe ejecutarse sin errores antes de entregarlo.
Suba su archivo (Notebook) a su cuenta de Github y comparta el link en la tarea de Canvas.

Herencia y super() (50 puntos)

Ejercicio 1 - Sistema de Empleados — **25 puntos**

Implementa la siguiente jerarquía de clases para un sistema de recursos humanos:

- Clase base Empleado(nombre, empresa, salario_base).
- Subclase GerenteVentas(nombre, empresa, salario_base, bono_ventas).
- Subclase ObreroPorHora(nombre, empresa, horas_trabajadas, valor_hora).

Cada clase debe implementar el método `calcular_salario()` y el método `descripcion()`. Usa `super()` en los constructores de las subclases.

Salida esperada

GerenteVentas: Ana Pérez - Salario: $2 300 000
ObreroPorHora: Pedro Soto - Salario: $640 000

Ejercicio 2 - Herencia Múltiple: Vehículos — **25 puntos**

Diseña una jerarquía de vehículos con herencia múltiple:

- Clases mixin: Electrico(bateria_kwh) y Combustion(cilindrada_cc).
- Clase base: Vehiculo(marca, modelo, año).
- Subclases concretas: Auto(Vehiculo), Moto(Vehiculo), AutoElectrico(Auto, Electrico).

Implementa el método `ficha_tecnica()` en cada clase. Verifica el MRO con `print(AutoElectrico.__mro__)`.

Clases Abstractas y Polimorfismo (50 puntos)

Ejercicio 3 - Sistema de Figuras Geométricas (ABC) — 25 puntos

Crea la clase abstracta `Figura(ABC)` con métodos abstractos `area()` y `perimetro()`. Implementa las subclases concretas:

- Circulo(radio)
- Rectangulo(base, altura)
- Triangulo(base, altura, hipotenusa)

Además, implementa la función polimórfica `calcular_total(figuras: list)` que retorne la suma de las áreas de todas las figuras.

Criterios de evaluación

- (5 pts) La clase Figura no puede instanciarse directamente.
- (5 pts) Cada subclase implementa correctamente area() y perimetro().
- (5 pts) describir() en la clase base reutiliza los métodos abstractos.
- (5 pts) calcular_total() es polimórfica - funciona con cualquier Figura.

Ejercicio 4 - Sistema de Pagos Polimórfico — 25 puntos

Diseña la interfaz abstracta `MetodoPago(ABC)` con métodos abstractos `procesar(monto)` y `reembolsar(monto)`. Implementa las subclases:

- PagoTarjeta(numero_tarjeta, titular)
- PagoTransferencia(cuenta_origen, cuenta_destino)
- PagoEfectivo()

Crea la función polimórfica `ejecutar_pago(metodo: MetodoPago, monto: float)` que procese el pago independiente del tipo.

2. Tabla de Puntaje

#	Ejercicio	Competencia evaluada	Nivel	Pts.
1	Sistema de Empleados	Herencia + super()	Nivel 1	25
2	Herencia Múltiple: Vehículos	MRO + mixins	Nivel 1	25
3	Figuras Geométricas (ABC)	Clases abstractas + polimorfismo	Nivel 2	25
4	Sistema de Pagos	Polimorfismo + duck typing	Nivel 2	25
			TOTAL	100

3. Notas Finales

Recuerda los principios clave trabajados en esta sesión:

- super() siempre antes de usar atributos heredados en el constructor.
- Una clase abstracta no puede instanciarse - define contratos, no objetos.
- El polimorfismo trabaja sobre la interfaz, no sobre el tipo concreto.